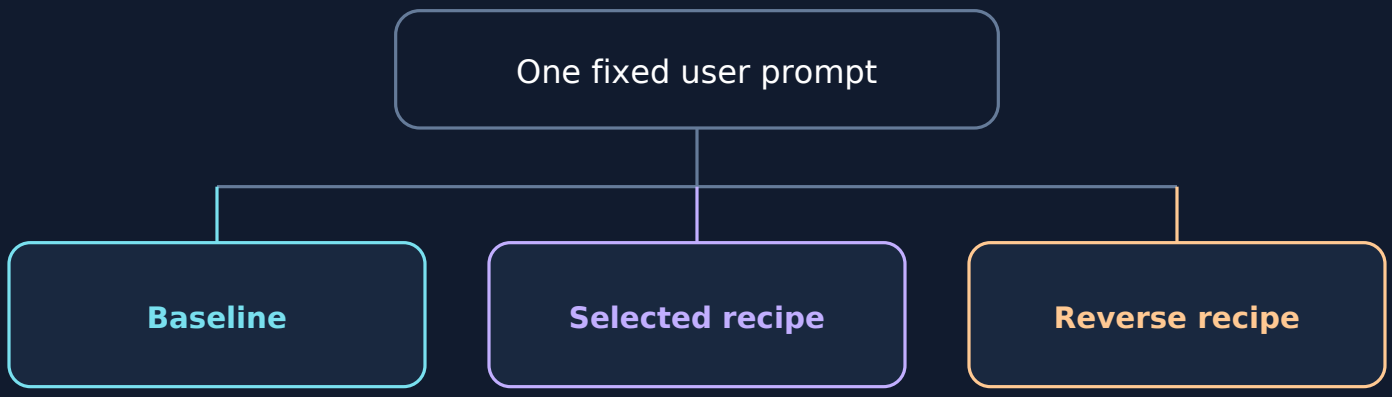
QWEN / NEURON LAB

One prompt. Different paths.

An investigation of Qwen3.5-4B neuron steering, a local M4 Pro implementation, and an interactive animation for Claude artifacts.



Source-verified implementation. Reproducible experiments.
BF16 reference + experimental GGUF port.
No unmeasured personality or hardware results are claimed.

Prepared for Rishabh Gupta

25 October 2026 / Apple M4 Pro / 48 GB unified memory

CHAPTER 01

What the investigation establishes

The supplied recipe describes a real, implementable activation intervention. The neuron-explorer repository is specifically built for Qwen3.5-4B and includes a live PyTorch runtime that selects Apple's MPS backend when available. Its code supports several simultaneous neuron pushes, not only one selected neuron. [1, 2]

That establishes implementation support. It does not establish that each supplied neuron reliably produces its informal personality label. The inspected research notes report other neuron effects and some Hugging Face versus vLLM agreement. They are the author's observations, not measurements made in this investigation and not validation of every supplied recipe. [3]

The deliverable has three paths:

Path	Purpose	Evidence status
Offline artifact	Explain the mechanism and inspect saved runs	Working source and local software checks
Hugging Face reference	Apply the literal upstream down_proj pre-hook	Source verified; full-model execution pending on your Mac
llama.cpp port	Apply equivalent-form residual vectors to BF16/Q8/Q4	Source-based algebra; runtime reproduction pending

No 4B model was downloaded or run in the authoring environment. No M4 throughput, memory peak, or personality result is claimed. The local environment had no PyTorch model runtime or Apple MPS device. Browser rendering was also unavailable because the browser binary download was blocked.

The project therefore ships an honest experiment, not a pre-established "evil model." Its offline example replies and probability values are visibly labelled authored or toy. A live backend replaces those with measured JSON outputs. Your next decisive step is a zero-dose run on the Mac, followed by individual-neuron dose tests.

CHAPTER 02

Steering, ablation, and abliteration

These terms describe different interventions. Keeping them separate makes the animation and the results easier to understand.

Operation	Mathematical idea	Persistent weight change?
Additive neuron steering	z[n] becomes z[n] + alpha	No
Neuron ablation	z[n] becomes 0	No, unless implemented as a weight edit
Multiplicative steering	z[n] becomes scale times z[n]	No
Directional ablation	Remove a projection from the residual	Usually an inference intervention
Abliteration	Commonly a weight edit that reduces a behavior-related direction	Usually yes
Quantization	Store weights with fewer numerical bits	Yes, in the saved representation

A negative additive dose is not ablation. If an activation is 0.2, adding -3 makes it -2.8; it does not make it zero. Negative MLP activations are meaningful because the gated activation can have either sign.

For a unit direction d_hat, directional ablation has the form:

```
r_new = r - d_hat * dot(d_hat, r)
```

A related weight-space approach projects output-writing matrices away from a direction. The appropriate matrices and side effects depend on the architecture; this is not achieved by simply typing a negative neuron dose.

Arditi and colleagues studied refusal-related directions and showed causal interventions in several models. That supports studying internal directions, but it does not prove a universal one-neuron personality map or validate these Qwen3.5 neuron indices. [8]

This repository implements additive inference steering and a residual-vector export. It does not permanently edit the checkpoint or claim to remove knowledge, train a personality, or complete an abliteration procedure.

CHAPTER 03

The exact model and neuron coordinates

Use the post-trained Hugging Face checkpoint Qwen/Qwen3.5-4B. Do not substitute Qwen3-4B, Qwen3.5-4B-Base, a 9B checkpoint, or an unrelated fine-tune while keeping these coordinates.

The official configuration describes a dense text backbone with 32 decoder layers, residual width 2,560, and MLP intermediate width 9,216. Three linear-attention layers are followed by a full-attention layer in each group of four. Full-attention indices are 3, 7, 11, 15, 19, 23, 27, and 31. [4]

A coordinate such as L14 N8949 means zero-based decoder layer 14 and gated-MLP activation index 8949. It is the fifteenth decoder block when counted in ordinary language. Every supplied neuron index fits the model's 0 through 9215 MLP range.

The upstream object path is:

```
model.model.language_model.layers[L].mlp.down_proj
```

The down-projection matrix has conceptual shape:

```
W_down: [2560 residual dimensions, 9216 MLP activations]
W_down[:, n]: one residual-space write direction
```

Calling a coordinate the "sarcasm neuron" is shorthand for an experimental association. A neuron can participate in several behaviors, and a behavior can depend on many neurons. A strong intervention can also create off-distribution text that resembles a style without representing a stable trait.

The source model contains a vision component as well. This project uses text prompts only. The reference runtime loads the conditional-generation model; the GGUF path uses the language-model file without requiring a vision projector for text chat.

CHAPTER 04

Deriving the supplied formula

Let u be the normalized MLP input. The gated MLP computes:

```
z = SiLU(W_gate * u) elementwise_times (W_up * u)
mlp_write = W_down * z
residual_out = residual_after_attention + mlp_write
```

For layer L and neuron n, define w = W_down[:, n]. Let R_L be the fixed median L2 norm of the unsteered residual leaving that decoder layer on calibration positions 1 onward. The requested dose d gives:

```
alpha = d * R_L / norm(w)
z_new[n] = z[n] + alpha
delta_residual = alpha * w
               = d * R_L * w / norm(w)
norm(delta_residual) = abs(d) * R_L
```

This normalization makes the individual write magnitude comparable across columns whose weight norms differ. Dose 1 means a write with the same norm as the calibration median residual. It is a substantial intervention, not "one unit" in the original activation's natural scale.

A small numeric example:

```
w = [1.2, 1.6]       norm(w) = 2
R = 10               dose = 0.6
alpha = 0.6 * 10 / 2 = 3
delta_residual = [3.6, 4.8]
norm(delta_residual) = 6 = 0.6 * 10
```

These numbers are a toy example. The actual write vector has 2,560 components.

For several neurons in one layer, add their vectors. The norm of the sum is not the sum of their norms: alignment can reinforce a push and opposition can cancel it. Across different layers, later computations are nonlinear, so mixture behavior cannot be predicted by adding separate personality descriptions.

CHAPTER 05

Calibration and hook placement

The upstream runtime registers a forward pre-hook on every layer's down_proj. At that point, the tensor contains the gated MLP activations, with shape approximately batch by sequence by 9216. Additive steering changes the selected component and passes the modified tensor into the linear down projection. [2]

Its residual measurement is after the complete decoder layer, including the MLP residual addition. The relevant distinction is:

Stage	Position 0 excluded?	Why
Compute R_L	Yes	Reduce the first-position sink's effect on calibration
Process the calibration input	No	Removing the token would move another token into position 0
Apply the steering push	No	The recipe intervenes at every input position
Decode the reply	No	Each newly processed token receives the push

The upstream code first looks for layer statistics and otherwise forwards a fixed reference passage. The fallback passage uses raw tokenization rather than a user chat template. It caches the resulting median. PyTorch's median behavior should be preserved when matching the implementation.

This wrapper uses a separate calibration_stats directory. By default it is empty, so the upstream fixed passage is used. If you deliberately add compatible upstream statistics, the exported row records the statistics file hash as the calibration source.

Calibrate with no intervention active and before enabling generation hooks. Do not recompute R from the already steered reply. Do not estimate a median from a single decode token. Do not confuse the MLP output norm, the normalized MLP input norm, and the residual norm leaving the layer.

The wrapper records R_L, column norm, alpha, individual write norm, source revision, and reference-text hash. Full replication of a social-media observation may still require its original checkpoint revision and calibration corpus.

CHAPTER 06

The supplied recipes

All values below are preserved in presets.json. UI intensity multiplies every listed dose. At intensity 1.00, the listed values are used exactly.

Preset	Layer, neuron, signed dose	Interpretation to test
Reported evil pair	L14 N8949 +1; L13 N7956 -1	Informal broad label; define a concrete behavior first
Sarcasm	L15 N3920 +1	Ironic or mocking phrasing
Sycophancy A	L12 N2920 -1	Agreement despite a false premise
Sycophancy B	L16 N8168 -1	Alternative candidate
Condolences	L10 N6295 +1	Negative framing of positive events
Chill friend	L11 N6261 +0.4; L17 N7870 -0.4	Warm, informal interaction
Mixture	L18 N5796 +0.6; L15 N3920 +0.6; L12 N2920 -0.5; L10 N6295 +0.4; L16 N8168 -0.3	Exploratory combined effect

Every interpretation is unverified in this deliverable. The labels came from the supplied recipe; the source investigation confirms that the intervention can be represented, not that these labels hold across prompts.

The reverse branch negates the complete recipe. For the reported pair, it applies L14 N8949 -1 and L13 N7956 +1 at intensity 1.00. It is not simply "turning the whole model down."

For a first experiment, select one neuron at intensity 0.25. Increase to 0.50 and then 1.00 while checking coherence. Test each mixture component separately before combining it. Preserve unchanged examples and failures.

CHAPTER 07

Why the same question can produce very different text

A language model predicts a distribution over the next token. Steering changes internal states that influence that distribution. Even with greedy decoding, a small change can swap the two highest-probability candidates.

The selected token then becomes part of the next input. If the baseline chooses "Congratulations" and the altered run chooses "Well," the next predictions already use different histories. Repeated steering plus this feedback can create a much larger difference in the final answer.

There are two distinct questions:

Question	Suitable evidence
Did the intervention change the next-token distribution?	Compare distributions at an identical prompt or teacher-forced prefix
Did the intervention cause a repeatable style change?	Score complete replies across held-out prompts, doses, and controls

The reference backend computes first-reply-token distributions at the same full prompt. It exports a small union of high-probability tokens and KL divergence in nats from baseline to each intervention. The KL calculation uses the full vocabulary, even though the frontend shows only a subset of tokens.

KL is directional. KL(baseline to recipe) need not equal KL(recipe to baseline). It measures distribution change, not sarcasm, honesty, or correctness.

The frontend also reports the first differing character in the returned text and a repetition measure based on word four-grams. These are descriptive measures. It does not invent token-level timelines from decoded strings.

A reply that "changes wildly" can indicate a useful causal effect, a change in style, or a breakdown in coherence. The evaluation must distinguish those outcomes.

CHAPTER 08

Mac setup and realistic memory expectations

Your M4 Pro's 48 GB is shared unified memory used by macOS, applications, and model workloads. A 4B-class model is a reasonable local target, but available free memory and runtime buffers matter.

Representation	Published GGUF file size	Use
BF16 text GGUF	8.42 GB	Precision comparison
Q8_0 text GGUF	4.48 GB	Intermediate comparison
Q4_K_M text GGUF	2.74 GB	Basic local quantization

These are the published file sizes in the selected Unsloth repository, not measured process memory. [5] The full Hugging Face conditional model also includes vision weights and runtime overhead. Context, three-branch batching, caches, and temporary tensors add memory. Plan for materially more memory than the weight file alone.

Start with short prompts and 96 to 160 new tokens. The reference runtime batches three conversations, so it can use more working memory than one chat. The GGUF runner instead loads one model at a time. Do not attempt the upstream all-neuron CUDA sweep on this Mac as an initial step.

Use ARM64 Python 3.11 or newer; Python 3.12 is a practical choice. Avoid mixing a Rosetta Intel Python environment with Apple Silicon packages. Use a current PyTorch build with MPS and BF16 support.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements-model.txt
python scripts/doctor.py
```

The doctor checks package imports, the Qwen3_5 class, MPS availability, and a small BF16 operation. It does not prove the full model will execute every operator. Do not report a CPU fallback run as an MPS benchmark.

CHAPTER 09

Run the literal Hugging Face intervention

This path follows the repository's original hook implementation and is the reference for interpreting the supplied neuron coordinates.

```
python scripts/setup_reference.py
python serve.py --backend reference
```

The setup script clones neuron-explorer at the audited commit and downloads the Hugging Face snapshot. It resolves the model's main revision once and writes the exact commit and local cache path to models/sources.lock.json. Later runs reuse that revision unless you supply another exact commit.

Open http://127.0.0.1:8792. Select sarcasm, use intensity 0.00 for the sham, and ask the same short question. Then try 0.25, 0.50, and 1.00. Model loading happens on the first real request.

Each comparison independently begins from the same single user message. There is no persona system prompt or inherited assistant history. The upstream runtime uses greedy argmax and disables thinking. Model weights remain unchanged.

The reference adapter returns three texts and common-prefix diagnostics. The server saves them with provenance under runs/. Export one JSON from the frontend and import it into the standalone artifact to present real results offline.

For the closest available dependency starting point, use the upstream locked environment:

```
cd vendor/neuron-explorer
uv sync --locked
cd ../..
vendor/neuron-explorer/.venv/bin/python scripts/setup_reference.py
vendor/neuron-explorer/.venv/bin/python serve.py --backend reference
```

Install uv separately if absent. The minimal requirements file is convenient, but it is not represented as a tested lock. Preserve package versions from the environment that actually succeeds on your Mac.

CHAPTER 10

Host a basic quantized model with llama.cpp

Install Xcode command line tools and CMake if needed. The setup script builds an audited llama.cpp source revision with Metal enabled on macOS, then downloads the named GGUF file from Hugging Face.

```
xcode-select --install
brew install cmake
python scripts/setup_llama.py --quant Q4_K_M
```

The selected download is unsloth/Qwen3.5-4B-GGUF, file Qwen3.5-4B-Q4_K_M.gguf. The script records the exact repository revision in models/Q4_K_M.lock.json. It downloads the ordinary language-model file for text use, not a multimodal projector.

For a basic chat service with llama.cpp's built-in web frontend:

```
vendor/llama.cpp/build/bin/llama-server \
  -m models/gguf/Qwen3.5-4B-Q4_K_M.gguf \
  --host 127.0.0.1 --port 8080 \
  -ngl 99 -c 4096 --parallel 1 \
  --jinja --reasoning off
```

Open http://127.0.0.1:8080. GPU layer offload is requested with -ngl 99; inspect the startup log to confirm Metal is actually used.

This ordinary server runs the baseline model. It does not accept arbitrary Python hooks or neuron-coordinate steering requests merely because it serves an OpenAI-compatible endpoint. Use exported control vectors and this project's comparison runner for the supplied additive recipes.

The pinned CLI supports --reasoning off. Older releases may use different options. Do not copy an older two-argument control-vector-scaled example into this source version: the inspected parser accepts FILE:SCALE syntax.

If using a prebuilt Homebrew binary instead, verify its version, Qwen3.5 support, control-vector behavior, and relevant flags before treating it as the audited runtime.

CHAPTER 11

Port the additive push to llama.cpp

The algebra follows from the down projection being linear:

```
W_down * (z + alpha * e_n)
  = W_down * z + alpha * W_down[:, n]
```

The audited Qwen3.5 graph applies its control vector after adding the MLP write to the residual. Therefore, with the same weights and calibration, adding alpha times the selected down column at that boundary reproduces the same mathematical residual change. [6]

This equivalence applies to the constant additive intervention. It does not turn multiplicative steering, activation clamping, or neuron ablation into a fixed control vector.

The exporter loads the BF16 source model, measures the fixed medians, and sums the selected write directions per layer. It writes float32 vectors for every supported layer, including zeros for unselected layers.

```
python scripts/export_vectors.py --all
python serve.py --backend llama \
  --gguf models/gguf/Qwen3.5-4B-Q4_K_M.gguf
```

Important indexing: the audited loader reads direction.14 into zero-based model layer 14, not layer 13 or 15. Layer 0 has no control-vector tensor in that loader. All supplied recipes use supported layers. The runner explicitly uses layer range 1 through 31. [7]

Each exported recipe bakes intensity 1.0. The CLI scale applies the UI intensity once, or its negative for reversal. Baseline has no vector.

The runner starts a fresh llama-server for baseline, recipe, and reverse, in sequence. This avoids reusing stale KV or recurrent state. It records commands, file hashes, usage, finish reasons, and elapsed time including each server load.

CHAPTER 12

What quantization changes

Quantization approximates weights. A Q4_K_M checkpoint uses a mixture of tensor encodings rather than storing every parameter as exactly four bits. Metadata and higher-precision tensors also affect file size.

If the original column is w and the quantized model effectively uses w_q, an activation push inside the quantized MLP would write alpha_q times w_q. This exporter instead uses the original BF16 column and its reference calibration to form a fixed residual direction.

Those operations are close only to the extent that their directions, norms, and surrounding computation agree. Thus, the exported vector is a controlled port of the original intervention, not a literal observation or mutation of a quantized neuron's activation.

Use this comparison ladder:

1. Hugging Face BF16 with the literal down_proj pre-hook. 2. A text BF16 GGUF with the exported residual direction. 3. Q8_0 GGUF with the same exported recipe. 4. Q4_K_M GGUF with the same exported recipe.

First check plain, unsteered behavior and prompt/template agreement between backends. Differences can come from the tokenizer template, thinking mode, numerical precision, or implementation, even before steering.

Do not require exact token equality across different backends as the only success criterion. Check zero-dose identity within each backend and measure behavioral retention across representations. A tiny numerical difference near an argmax tie can create a different continuation.

An advanced extension could extract dequantized GGUF columns and recalibrate residual norms in that backend. That would more closely target its own activations. This project does not implement that extension, and does not present its BF16-derived calibration as a GGUF measurement.

CHAPTER 13

Reproducible evaluation

Start with one question in the UI. Then run the included benign prompt battery:

```
python scripts/run_experiments.py \
  --preset sarcasm --doses 0,0.25,0.5,1 --repeat 2
```

There are 12 prompts and three branches per comparison. The command performs 96 comparisons, or up to 288 reply branches, so it is not a quick smoke test. Use shorter custom batteries when iterating.

The included controls are zero-dose sham, reverse sign, repeated baselines, multiple doses, and common-prefix diagnostics on the reference path. Norm-matched random directions and adjacent-neuron controls are recommended extensions, not implemented features.

Observation	What it supports	What it does not prove
Zero-dose replies match	Basic branch consistency	Personality specificity
Next-token KL increases	A distribution changes at the prompt boundary	Better, worse, or safer behavior
Sarcasm scores increase	A repeatable style association on that prompt set	One universal sarcasm neuron
False premises are endorsed more often	Increased agreement on that test	General sycophancy in all settings
Repetition increases	Possible degradation at that dose	A successful personality change

Blind the branch labels for scoring. Evaluate sarcasm, false-premise correction, inappropriate condolences, warmth, factual accuracy, instruction following, and coherence separately. Define success before viewing the outcomes.

Preserve negative results. "It did not reproduce under this model revision and calibration" is a valid result. The project's research value comes from traceable conditions and a clear denominator, not from selecting the most dramatic reply.

CHAPTER 14

Animation and Claude artifact design

The supplied artifact is one offline HTML file. It uses no external fonts, JavaScript packages, or CDNs. Its five stages are experiment, MLP mechanism, autoregressive divergence, replies, and evidence/setup.

Use a dark navy canvas with white body text. Baseline is cyan, the selected recipe is violet, and reversal is warm orange. Keep body text at 16 to 18 pixels, card padding at 24 to 32 pixels on desktop, and one primary diagram per stage.

The mechanism animation highlights one actual operation at a time. It has manual play/pause and stepping. Reduced-motion users get step changes rather than continuous animation. It stops when the reader leaves that stage.

The toy vector is explicitly two-dimensional and uses made-up values. The real residual is 2,560-dimensional; a two-dimensional picture is not a measured circuit map. Arrow widths and playback timing do not represent signal strength or model latency.

The reply stage initially contains labelled authored examples. It must keep those labels until a real backend run or a measured JSON import replaces them. The probability table follows the same rule. GGUF runs do not acquire fake probability bars merely because the reference backend can export them.

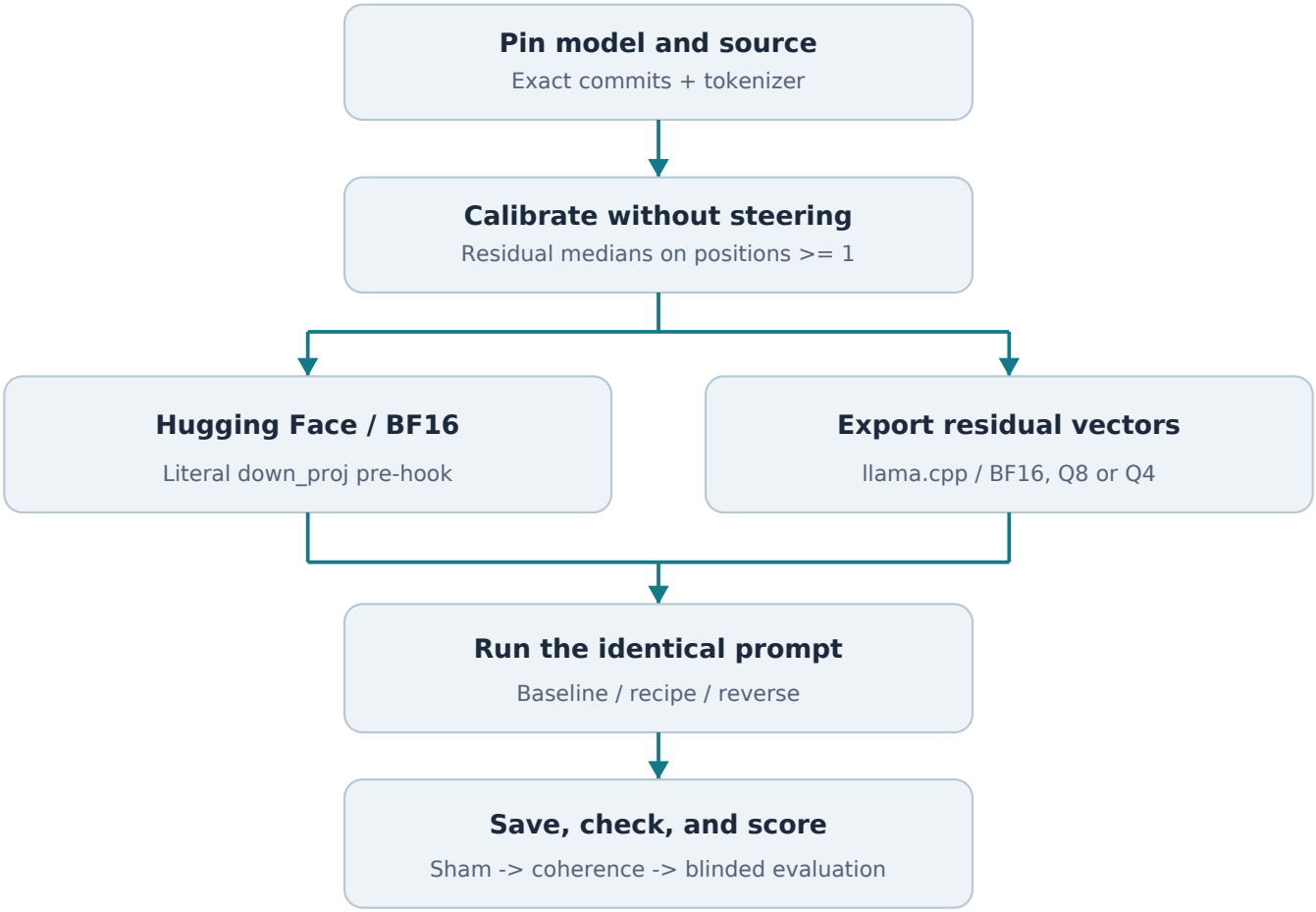
Claude Code can modify and run a local project. Claude.ai artifacts run in a managed preview that may not reach localhost or allow arbitrary packages. The artifact's JSON-import path supports presenting real runs there without assuming a live connection.

Use docs/CLAUDE_ARTIFACT_PROMPT.md for a single-artifact request. Use docs/CLAUDE_CODE_PROMPT.md for source verification, execution, evaluation, and design work on the Mac.

CHAPTER 15

End-to-end workflow

The workflow separates calibration, execution, and interpretation. It also separates the literal neuron intervention from its GGUF port.



Read the diagrams in docs/WORKFLOW.md for copyable Mermaid source. The branches converge on the same prompt and output schema so results can be compared without disguising the backend differences.

If a zero-dose test fails, inspect the prompt template, thinking mode, hooks, batching, numerical reproducibility, and cache state before assigning a semantic interpretation. If a higher dose creates loops or empty replies, reduce the dose and record the failure.

A successful software run is not automatically a successful behavioral reproduction. The final evaluation should name the exact model revision, calibration source, backend, representation, prompt set, and criterion.

CHAPTER 16

Use the original 3D neuron-explorer frontend

This project provides a compact educational frontend. The upstream repository provides the larger exploration interface, including a 3D model map, neuron lists, lenses, connections, and live steering. [1]

To run the original frontend with its own environment:

```
cd vendor/neuron-explorer
uv sync --locked
uv run python explorer/server.py
```

Open http://localhost:8791. Use a different terminal for this project's frontend at port 8792.

The original server can load another copy of the model when its live tools are used. On the Mac, run one inference service at a time until you understand memory use. Merely opening cached views is different from requesting a live model operation.

Its default data layout expects runs/, stats/, and explorer/cache/ in its own repository. Some caches are included; the full sweep rollouts, statistics, captures, and embeddings are separate research data. Missing panels do not mean the neuron coordinates are invalid. Do not claim that a normal clone contains every research run.

The upstream server's agent steering endpoint accepts a neurons list with layer, n, dose, and mode. Its Runtime.steer_multi is the method wrapped here. This avoids assuming that a single-neuron slider in the original interface can directly express the whole supplied mixture.

The upstream Claude agent panel is optional and requires its own installed, authenticated Claude CLI. It is not necessary for local steering. The original server also implements local-origin checks because that panel exposes tools beyond visualization. Keep it bound locally.

CHAPTER 17

Troubleshooting and common mistakes

Symptom	Likely check or action
Qwen3_5 class cannot import	Use compatible current Transformers or the upstream locked environment; Qwen3 and Qwen3.5 are different
Model runs slowly	Check doctor output and startup logs for MPS/Metal versus CPU
MPS operator error	Confirm current ARM64 packages; test an explicit CPU diagnostic separately
Memory pressure	Close extra services, shorten prompts, lower output length, and use GGUF for the lightweight path
Preset vector missing	Run export_vectors.py for that preset before the llama comparison
Port 8081 already used	Stop the old process or pass another --llama-port
No next-token bars for GGUF	Expected: this backend does not export those diagnostics
Changed UI recipe but old replies	Run again; the displayed replies retain their original saved-run settings
Upstream map lacks rollouts	Obtain the missing runs/stats or use its live tools
High-dose loops or broken language	Record the result and lower intensity; do not treat breakdown as a validated style

Start with the unsteered model. If it cannot answer a short question, neuron steering is not the first problem to debug.

Do not use bitsandbytes CUDA-only recipes as the assumed Apple Silicon solution. This project uses full BF16 PyTorch for the reference and GGUF/Metal for compact hosting. MLX is another possible Apple runtime, but the supplied wrapper does not pretend that an MLX quantized model supports these exact PyTorch hooks.

Preserve new errors and the package environment in runs/. Do not overwrite the known source pins merely to silence an error. If you update them, review the hook location, graph boundary, CLI syntax, and layer mapping again.

The setup scripts clone source and download model files. They do not create a public service, upload your prompts, or publish a GitHub repository.

CHAPTER 18

Deliverables, validation, and next step

The repository includes a standard-library server, two live backend adapters, all recipe presets, model setup scripts, a float32 control-vector exporter, a repeated experiment runner, a standalone artifact, Claude prompts, a mathematical guide, Mermaid diagrams, and tests.

The model files are intentionally not packaged. Downloads use Hugging Face, and source checkouts use the pinned GitHub revisions. The archive therefore stays small and can be inspected before any model download.

Validation is divided into two categories:

Completed locally	Still requires the target runtime
Python syntax and request validation	Full 4B reference inference
Mathematical vector equivalence on synthetic arrays	Zero-dose identity with the real model
GGUF packaging with the official reader	Actual llama.cpp control-vector generation
Local HTTP integration and error handling	MPS memory, speed, and operator compatibility
Frontend JavaScript syntax and structural checks	Browser visual checks and Claude.ai preview
PDF rendering and visual inspection	Behavioral reproduction of the named recipes

See docs/VALIDATION.md for the actual test results. No local source or unit test substitutes for the remaining hardware and model checks.

Recommended first run: start illustration mode to inspect the interface, set up the reference model, run intensity 0.00, then test L15 N3920 at 0.25 and 0.50 on the same positive-event prompt. Export the real JSON and import it into the artifact.

After a successful reference run, export the vectors, allow that process to exit, and repeat with Q8_0 or Q4_K_M. That sequence will answer both whether a supplied recipe reproduces and how much its behavior survives quantization.

CHAPTER 19

Primary sources and audit pins

[1] Neuron-explorer README and repository tree. Audited source commit f79b2196e29aacbad89684938baac8d97b172fd1, committed 24 October 2026 UTC. https://github.com/lyramakesmusic/neuron-explorer

[2] Upstream live runtime: pre-hook, residual calibration, multi-neuron generation, and next-token log-probabilities. https://github.com/lyramakesmusic/neuron-explorer/blob/f79b2196e29aacbad89684938baac8d97b172fd1/explorer/runtime.py

[3] Upstream research log: author-reported experiments, limitations, and engine comparisons. https://github.com/lyramakesmusic/neuron-explorer/blob/f79b2196e29aacbad89684938baac8d97b172fd1/research_log.md

[4] Qwen model card and configuration. https://huggingface.co/Qwen/Qwen3.5-4B https://huggingface.co/Qwen/Qwen3.5-4B/blob/main/config.json

[5] Selected GGUF distribution and published file sizes. https://huggingface.co/unsloth/Qwen3.5-4B-GGUF/tree/main

[6] llama.cpp Qwen3.5 graph, audited commit d854d98bae598388ecea1ed84396587e5edc326d. https://github.com/ggml-org/llama.cpp/blob/d854d98bae598388ecea1ed84396587e5edc326d/src/models/qwen35.cpp

[7] Control-vector loader, layer mapping, and addition. https://github.com/ggml-org/llama.cpp/blob/d854d98bae598388ecea1ed84396587e5edc326d/src/llama-adapter.cpp https://github.com/ggml-org/llama.cpp/blob/d854d98bae598388ecea1ed84396587e5edc326d/common/common.cpp

[8] Arditi et al. Refusal in Language Models Is Mediated by a Single Direction, 2024. https://arxiv.org/abs/2406.11717

[9] Hugging Face Transformers Qwen3.5 documentation. https://huggingface.co/docs/transformers/main/en/model_doc/qwen3_5

The GitHub pins are source-audit anchors, not proof that every dependency was executed. The model setup scripts resolve and preserve concrete Hugging Face revisions on the Mac. Upstream source has not been copied into the deliverable, and its redistribution license was not assumed.